

INF201 - Lecture 2

14th Sept 2026

Topics

- Mandatory assignments
- Organising your work in folders
- Doc-strings
- Formatting strings
- Python data types
- Dynamic vs static typing
- Type hints
- A deep dive into lists

Rules for in-class exercises and assignment submissions

- Week 37-40
- You work on these throughout the week, especially during the exercise classes (Groups 1-7)
- Talk to the teaching assistant(s) in class, as attendance is mandatory
- You work in pre-assigned groups of 3-4 people, where you can discuss your code and learn from each other
- You must upload your code (and AI chat-log if relevant) to Canvas by the deadline on Friday
 - Please do not submit zipped folders
 - You may submit Python scripts (.py) or Jupyter notebook files (.ipynb)
 - If you use Jupyter notebook, please also submit a pdf version
 - You can either export to pdf directly from VS Code (requires TeX)
 - Alternatively you can export it as html, and create a pdf in your web browser
 - [Jupyter Notebook in VS Code tutorial](#)
 - The AI chat log can be submitted as a text document, Word (or similar) file or pdf
 - If the chat log is long, you may instead submit a summary
 - Suggested prompt: "Could you list the questions I have asked you and give a short summary of our conversation?"
- Even if you have worked together on your code, you must all upload separately on Canvas
- You will be rewarded up to 5 points per assignment

- The teaching assistants will give you points based on what you upload to Canvas if (and only if) you also showed up to the class

The assignment is too vague, I am confused?!?

- Some assignments will not be as clearly defined as others
- This forces you to think through what you want your script to do and which assumptions you are making
- You have more creative freedom here
- The assignment will state what we require to give you points

Organising your work into folders

- VS Code requires you to work from a folder
- The first time you use a folder, you must tell VS Code that you trust it
- VS Code will look for a .venv sub-folder and (if found) automatically activate the venv
- VS Code will also apply settings found in a .vscode/settings.json or a pyproject.toml
- Organising your work and files in well-named folders is a good habit for any programmer
- Do yourself a favour and set up a tidy and organised folder system!
- I will show an example in the lecture

Formatted string literals: f-strings

- A very flexible way to format strings in Python

```
In [6]: # Printing or writing nice looking strings with a lot of flexibility
name = "Turid"
course = "INF201"
year = 2026
print("Hello, " + name + "! You are teaching " + course + " the autumn of " + str(y
print(f"Hello, {name}! You are teaching {course} the autumn of {year}.")

# Formatting numbers in strings
number = 2
print(f"{number} divided by 3 is {number / 3}.")
print(f"{number} divided by 3 is {number / 3:.2f}.")
print(f"{number} divided by 3 is {number / 3:.1%}.")

# Alignment and padding
print(f"{42:0>5}")
print(f"{'Lower case':<15}{'Upper case':<15}")
for i in range(5):
    lower = chr(97 + i)
    upper = chr(65 + i)
    print(f"{lower:<15}{upper:<15}")
```

```

Hello, Turid! You are teaching INF201 the autumn of 2026.
Hello, Turid! You are teaching INF201 the autumn of 2026.
2 divided by 3 is 0.6666666666666666.
2 divided by 3 is 0.67.
2 divided by 3 is 66.7%.
00042
Lower case      Upper case
a               A
b               B
c               C
d               D
e               E

```

Doc-strings

- [PEP 257 – Docstring Conventions](#)
- These are comments used to document functions
- Starts on the first line after the function definition
- Written with triple quotes `"""`
- On-liner if the function is fairly obvious
- Can also list argument(s) and return(s)

```

In [7]: def add_ints(x: int, y: int) -> int:
        """Add two integers together."""
        return x + y

def process_class_scores(scores: list[float], pass_threshold: float = 50.0) -> float:
    """
    Returns the average score of passing students.

    Args:
        scores: List of student scores.
        pass_threshold: Score threshold for passing. Default is 50.0.
    Returns:
        Average score of passing students. Returns 0.0 if no students passed.
    """

```

Data types in Python

The following is an incomplete list of [built-in data types in Python](#)

- Numeric types
 - int `3`
 - float `3.14`
 - complex `3+4j`
- Sequence types
 - list `[1, "a", 3.14]`
 - tuple `(1, "a", 3.14)`

- `range` `range(5)`
- Text sequence types `*string` `"text"`
- Set types `*set` `{1, "a", 3}`
- Mapping types
 - `dict` `{"a" : 1, "b": 2}`
- Special types
 - `NoneType` `None`
- Boolean types
 - `bool` `True`, `False`

Dynamic vs static typing

Python is dynamically typed

- We do not have to tell Python which data type a variable has
- Type is inferred when we run the code
- A variable can change type without problems


```
a = 3 # a in an integer
a = "text" # a is now a string
```
- This would not be the case in a statically typed language


```
// C code
int x = 4;
```

Comparison of static and dynamic typing

- Dynamic typing
 - Examples: Python, Ruby
 - Type is determined at run-time
 - A variable is a label or reference to an object in memory
 - Faster and easier to write
 - Unexpected type errors when running code
- Static typing
 - Examples: C, Java
 - Type is determined at compile-time
 - Variables are typed "boxes" (locations) in memory
 - Type errors are caught during compilation
 - Code runs faster
- The Whilwind Tour of Python has a [good chapter](#) introducing this concept
- (<https://nedbatchelder.com/text/names>) is a excellent and more thorough discussion of this (read it!)

Beware of the aliasing trap

```
In [8]: x = [1, 2, 3]
        y = x
        y.append(4)
        print(y) # Output: [1, 2, 3, 4]
        print(x)
        print(id(x))
        print(id(y))
        print(id(x) == id(y))
```

```
[1, 2, 3, 4]
[1, 2, 3, 4]
1649437734400
1649437734400
True
```

- `x` and `y` point to the same object, as shown by their identical memory address
- `x` and `y` are aliases of the same object in memory!
- Changing `y` therefore changes `x` as well
- [Visual demo on this in Python Tutor](#)

Type hints

- Type hints are a way to help avoid typing errors in Python
- Hints about which data type we expect the function input and output to have
`def process_class_scores(scores: list[float], pass_threshold: float = 50.0) -> float:`
- `list[float]` indicates that we expect the input score to be a list of floating-point numbers
- `-> float` indicates that we expect the function to return a floating-point number

```
In [9]: def add_no_hints(x, y):
        """Add two numbers together."""
        return x + y

def add_with_hints(x: int | float, y: int | float) -> int | float:
    """Add two numbers together."""
    return x + y

print(add_no_hints(1, 2)) # Output: 3
print(add_no_hints(1.5, 2.5)) # Output: 4.0
print(add_no_hints("hello", " world")) # Output: "hello world"

print(add_with_hints(1, 2))
print(add_with_hints(1.5, 2.5))
print(add_with_hints("hello", " world"))
```

```

3
4.0
hello world
3
4.0
hello world

```

- Note that the type hints are not enforced!
- We can still input strings to the function that has int and float as type hints
- We should still use type hints when writing Python functions
- They help document the function
- They force you to think through which data types you expect
- They can be used by type checkers like `mypy` to identify potential errors early
- [PEP484](#) discuss type hints
- See also chapter 2 of (<https://link.springer.com/book/10.1007/978-3-031-40336-1>)

Common type hint patterns

- Single type


```
def add_ints(x: int, y: int) -> int:
    """Add two integers together."""
    return x + y
```
- Multiple types'


```
def add(x: int | float, y: int | float) -> int | float:
    """Add two numbers together."""
    return x + y
```
- Containers


```
def calculate_grade_stats(
    student_list: list[dict[str, str]], pass_threshold: float = 50.0
) -> dict[str, Any]:
```
- Special types
 - `Any`
 - `None`

```
def my_print_function(x: int | float | str) -> None:
    """Print a value."""
    print(x)
```

A deep dive into lists

- A list is a very useful container in Python
- A container can hold and structure a collection of other objects
- Sets, tuples and dicts are some other examples of containers
- A list is a sequence, meaning that the order of elements matter
- It is random access; you can access any element you want in any order
- It is mutable, meaning that it can be changed

Copying lists

```
In [10]: list1 = [1, 2, 3, 4, 5]
list2 = list1

print(f"list1 is at {id(list1)}")
print(f"list2 is at {id(list2)}")
```

```
list1 is at 1649438224896
list2 is at 1649438224896
```

- `list1` and `list2` are aliases of the same object
- We can make a shallow copy of a list by using `.copy()` or by slicing `[:]`

```
In [11]: list3 = list1.copy()
print(f"list3 is at {id(list3)}")

list4 = list1[:]
print(f"list4 is at {id(list4)}")
```

```
list3 is at 1649438383680
list4 is at 1649438164416
```

- These now represent different object in memory
- What happens if we have a nested list?

```
In [12]: nested_list1 = [1, 2, [3, 4], 5]
print(f"nested_list1 is at {id(nested_list1)}")

nested_list2 = nested_list1.copy()
print(f"nested_list2 is at {id(nested_list2)}")

nested_list2[2][0] = 0
print(nested_list2)
print(nested_list1)
```

```
nested_list1 is at 1649438023424
nested_list2 is at 1649438029312
[1, 2, [0, 4], 5]
[1, 2, [0, 4], 5]
```

- They are different object, but changing one still changed the other. What happened?
- A shallow copy duplicates the outer container but leaves the inner elements pointing to the same original objects in memory

```
In [13]: print(f"nested_list1 element 2 is at {id(nested_list1[2])}")
print(f"nested_list2 element 2 is at {id(nested_list2[2])}")
```

```
nested_list1 element 2 is at 1649438017408
nested_list2 element 2 is at 1649438017408
```

- To get around this you can `import deepcopy from copy`

- See visualisation in [Python Tutor](#)